Load Testing (CRM CASA 360 / Customer 360, CBS Cash Deposit / Balance Enquiry)

Browser-based load test harness built on Playwright. Each virtual user (VU) is a real Chromium session that authenticates **exactly once** and then loops the business flow over its own data records until the test duration or iteration cap is reached. Every business step is timed as a named transaction, and all samples are aggregated into an industry standard report (percentiles, throughput, error rate, Apdex, SLO verdict, time series).

1. Prepare test data

Test data lives in CSV files, one row per data set. Rows are **grouped by `username`/`password`**: a VU is assigned one user group, logs in once as that user and then cycles round-robin only through that user's records. This guarantees the session and the data always belong to the same user.

- `fixtures/casa360.csv` — `username,password,casaNumber`
- `fixtures/customer360.csv` — `username,password,customerId`
- `fixtures/crm_deposit360.csv` — `username,password,depositNumber`
- `fixtures/crm_srMobileUpdate.csv` — `username,password,customerId,newMobileNumber,srType,category,subCategory,description,filePath`
- `fixtures/aoco_cifAlone.csv` — `username,password,constitution`
- `fixtures/cbs_balanceEnquiry.csv` — `username,password,customerId`
- `fixtures/cbs_cashDeposit.csv` — `username,password,accountNo,amount,narrative,noteDenomination,authorizer`
- `fixtures/cbs_cashWithdrawal.csv` — `username,password,accountNo,amount,noteDenomination,authorizer`
- `fixtures/cbs_casaTransfer.csv` — `username,password,fromAccountNo,amount,toAccountNo,authorizer`
- `fixtures/cbs_neftPayment.csv` — `username,password,accountNo,amount,beneAccountNo,beneIfsc`
- `fixtures/cbs_rtgsPayment.csv` — `username,password,casaAccountNo,amount,beneIfsc,beneAccountNo`
- `fixtures/cbs_impsPayment.csv` — `username,password,accountNo,amount`
- `fixtures/cbs_loanPayment.csv` — `username,password,loanAccountNo,casaAccountNo,amount,authorizer`
- `fixtures/cbs_tdQuickPayin.csv` — `username,password,accountNo,glAccountNo,amount,termMonths,termDays,authorizer`
- `fixtures/cbs_tdRedemption.csv` — `username,password,tdAccountNo,casaAccountNo,authorizer`
- `fixtures/cbs_nomineeMaintenance.csv` — `username,password,accountNo,nomineeName,dateOfBirth,sharePercentage`
- `fixtures/cbs_passbookReissue.csv` — `username,password,accountNo,newPassbookNo`

Lines starting with `#` are comments. Values may be quoted if they contain commas. Every row must carry `username` and `password`.

> Use distinct application users where possible. VUs are assigned users round-robin, so with 25 VUs and 2 users each user carries ~13 concurrent sessions, which can serialise server-side state and distort results. Aim for one user per VU.

Override any path with `<FLOW>_DATA`, e.g. `CASA360_DATA`, `CASHDEPOSIT_DATA`, `TDREDEMPTION_DATA`.

Cash deposit / withdrawal data rules

`amount` must be exactly divisible by `noteDenomination`; the note count handed to the denomination screen is derived as `amount / noteDenomination` (see `utils/cbs.helpers.js`). A mismatch is rejected before the run starts rather than producing a flood of misleading CBS validation errors.

Service request data

For `srMobileUpdate`, `filePath` is the attachment uploaded on the SR form and is resolved relative to the repository root. `srType`, `category` and `subCategory` must match the CRM dropdown labels exactly.

Single-use data

- **`tdRedemption`** — a full redemption closes the deposit, so seed one TD per planned iteration.
- **`nomineeMaintenance`** — an account only accepts a limited number of nominees; give each user its own account.
- **`passbookReissue`** — the spec appends VU and iteration to `newPassbookNo`, so the CSV value acts as a prefix and stays unique per attempt.

> **`srMobileUpdate` and `aocoCifAlone` write data.** Each iteration saves a real service request / opens a real onboarding application.

> **Every CBS flow except `balanceEnquiry` writes data.** Each iteration posts and authorises a real transaction or maintains customer data. A 10 VU / 10 minute run can create thousands of postings and will move teller till and account balances. Only run against UAT/DR with explicit sign-off, and agree the data clean-up beforehand.

2. Run

```
# validation run first: 2 VUs per flow, 2 iterations each
npm run load:smoke

# full run with the configured profile
npx cross-env LOAD_TEST=1 VUS=25 RAMP_UP_SEC=300 DURATION_SEC=1800 npm run load

# whole application: SYSTEMS expands to every flow of that app and the folder
# scopes collection to its specs
npm run load:crm
npm run load:cbs
npm run load:aoco
npm run load:crm:smoke              # 1 VU, 1 iteration per flow
```

```
npm run load:cbs:smoke
npm run load:aoco:smoke

# single flow
npm run load:casa360
npm run load:customer360
npm run load:deposit360

# CRM service request — saves a real SR per iteration
npm run load:srmobileupdate

# AOCO onboarding — opens a real application per iteration
npm run load:aoco:cif:smoke         # 1 VU, 1 application, validates the flow
npm run load:aoco:cif

# CBS balance enquiry — read-only, safe to run at higher concurrency
npm run load:balanceenquiry:smoke
npm run load:balanceenquiry

# CBS cash deposit — posts real transactions, never included by default
npm run load:cashdeposit:smoke      # 1 VU, 1 deposit, validates the flow end to end
npx cross-env LOAD_TEST=1 FLOWS=cashDeposit VUS=5 RAMP_UP_SEC=60 DURATION_SEC=600 \
  npx playwright test tests/cbs

# the remaining CBS flows — all write data
npm run load:cashwithdrawal
npm run load:casatransfer
npm run load:neft
npm run load:rtgs
npm run load:imps
npm run load:loanpayment
npm run load:tdquickpayin
npm run load:tdredemption
npm run load:nominee
npm run load:passbook

# mixed CBS load: read-heavy enquiries alongside a lighter posting load
npx cross-env LOAD_TEST=1 FLOWS=balanceEnquiry,cashDeposit VUS_BALANCEENQUIRY=8 VUS_CASHDEPOSIT=2 \
  RAMP_UP_SEC=60 DURATION_SEC=600 npx playwright test tests
```

`npm run load` targets `tests/load` only. Regular functional runs (`npm test`) exclude the load specs automatically.

Folders vs selection

Load specs live under `tests/load/<app>/` (`crm`, `cbs`, `aoco`). The folder only scopes which spec files Playwright collects; a flow still runs only when it is selected:

- `SYSTEMS=cbs` — all flows of that application, write flows included.
- `FLOWS=cashDeposit` — individual flows.
- Additive: `SYSTEMS=cbs FLOWS=casa360` runs all CBS flows plus CRM CASA 360 (use the parent `tests/load` path in that case).

Running a folder without either variable lists every spec as `disabled via FLOWS`.

3. Configuration (all environment variables)

Variable	Default	Meaning
`BASE_URL`	`https://dr-crm.esafbank.org`	CRM entry point under test
`TEST_ENV`	`dr`	Free-text environment label stamped on the repo
`CBS_BASE_URL`	`https://dr-cbs-uat-wll.esafbank.org:31108/obxlogin/login.html`	CBS entry point under test
`AOCO_BASE_URL`	`https://dr-onboardingapp.esafbank.org/omniapp/pages/login/loginapp.app`	AOCO onboarding entry point under test
`SYSTEMS`	–	Comma separated applications to run in full: `crm`
`FLOWS`	`casa360,customer360`	Comma separated flows (additive with `SYSTEMS` `customer360`, `deposit360`, `srMobileUpdate` `aocoCifAlone`, `balanceEnquiry`, `cashDepos` `cashWithdrawal`, `casaTransfer`, `neftPayme` `rtgsPayment`, `impsPayment`, `loanPayment`, `t` `tdRedemption`, `nomineeMaintenance`, `passb`
`VUS`	`10`	Virtual users per flow

Variable	Default	Meaning
`VUS_<FLOW>`	`VUS`	Per-flow VU override, e.g. `VUS_CASHDEPOSIT=2` `VUS_BALANCEENQUIRY=8`
`<FLOW>_DATA`	per flow	CSV path override, e.g. `NEFTPAYMENT_DATA=./`
`<FLOW>_TASK_CODE` / `<FLOW>_TASK_LABEL`	per flow	CBS task to open — `1401`/`1001`/`1091`/`2057`/`2055`/`7022`/`1021`/`7201`/`B`
`RAMP_UP_SEC`	`60`	VUs are started linearly across this window
`DURATION_SEC`	`600`	Steady-state duration per VU
`ITERATIONS`	`0` (duration driven)	Hard cap on iterations per VU
`THINK_TIME_MIN_MS` / `THINK_TIME_MAX_MS`	`1000` / `3000`	Random pause between iterations
`HEADLESS`	`true`	Set `false` to watch the browsers
`WORKERS`	`= total VUs`	Playwright worker processes (one browser each)
`ABORT_ON_ERROR`	`false`	Fail the whole VU on the first transaction error
`RECOVER_ON_FAILURE`	`true`	After a failed iteration, clear blocking popups an tabs before the next one
`MAX_CONSECUTIVE_FAILURES`	`3`	End a VU after this many failed iterations in a ro
`ACTION_TIMEOUT_MS`	`60000`	Playwright timeout for a single click/fill
`NAV_TIMEOUT_MS`	`90000`	Playwright navigation timeout
`FIELD_TIMEOUT_MS`	`10000`	Wait for a server-populated field (rate, derived a
`TEST_TIMEOUT_MS`	derived from ramp-up + duration	Hard override for the per-VU test timeout
`APDEX_T_MS`	`3000`	Apdex satisfied threshold (tolerating = 4T)
`SLO_P95_MS`	`5000`	SLO: p95 response time
`SLO_P99_MS`	`8000`	SLO: p99 response time
`SLO_ERROR_RATE_PCT`	`2`	SLO: maximum error rate
`SLO_MIN_TPS`	`0` (disabled)	SLO: minimum throughput
`RESULTS_DIR`	`./load-results`	Root output folder
`LOAD_RUN_ID`	timestamp	Run identifier / output sub-folder
`REPORT_BUCKET_SEC`	`10`	Time-series bucket width

Capacity note

One VU is one Chromium instance. Budget roughly 250–400 MB RAM and ~0.3 vCPU per VU. Beyond ~20–25 VUs on a laptop, the client becomes the bottleneck and the numbers measure your machine, not the CRM. Distribute across agents for higher concurrency.

4. Transactions measured

CASA 360 — `00 Login`, `01 Search account`, `02 Open account details`, `03 Financial Details tab`

Customer 360 — `00 Login`, `01 Search customer`, `02 Open customer details`, `03 Communication tab`, `04 KYC Info tab`

CBS Cash Deposit — `00 Login`, `01 Open task 1401`, `02 Enter transaction`, `03 Denomination details`, `04 Authorise`

CBS Balance Enquiry — `00 Login`, `01 Open task 7100`, `02 Query customer balances`, `03 Back to enquiry list`. Read-only, so this is the flow to use for sustained, high-concurrency CBS capacity tests.

Session model

`00 Login` is executed once per VU, before the iteration loop, and is recorded as its own transaction so it never inflates the per-iteration numbers. The session is then reused for every iteration.

After a failed iteration the harness checks whether the CRM login form has reappeared:

- **Session alive** — nothing happens, the next iteration continues on the same session.
- **Session lost** — a `00 Re-login (session recovery)` transaction is recorded and the VU re-authenticates. If that also fails, the VU ends.

Re-login counts are printed per VU at the end of the run and visible as a separate row in `transactions.csv`. A non-zero count is a finding in itself: it usually means the server dropped sessions under load.

CBS additionally signs out once per VU after the last iteration, so teller sessions are released cleanly instead of being killed with the browser.

CBS login hands over to the Oracle Flexcube Retail **popup window**. A flow's `login` returns that page and the harness then runs every transaction, screenshot, navigation timing and HTTP metric against it (see `utils/cbsSession.js`).

Each transaction ends only once the UI has stopped mutating (`waitForUiIdle`), so the timing covers server response plus rendering. Additional synthetic samples are written per iteration: `ITERATION (end-to-end)`, `PAGE LOAD (navigation timing)` and `HTTP (network summary)`.

5. Outputs

Everything lands in `load-results/<runId>/`:

Artefact	Contents
`index.html`	Self-contained dashboard: KPIs, SLO verdict, trend charts, all tables
`summary.json`	Full machine-readable report (CI gates, trend databases)
`transactions.csv`	Per transaction: count, pass/fail, min/avg/median/p90/p95/p99/max, std dev, TPS, Apdex
`flows.csv`	Same statistics rolled up per flow
`timeseries.csv`	Per interval: active VUs, throughput, errors, avg and p95 response time
`samples.csv`	Raw sample log, one row per transaction execution
`errors.csv`	Grouped failures with counts and first/last seen
`http-metrics.csv`	Requests, 4xx, 5xx, failed requests, MB received, slowest request
`browser-metrics.csv`	TTFB, DOM interactive, DCL, load, FCP (avg and p95)
`screenshots/`	Screenshot per failed iteration
`raw/*.jsonl`	Per-VU sample streams (source of truth for aggregation)
`playwright-results.json`, `playwright-html/`	Native Playwright execution report

The report is generated automatically in the global teardown. To rebuild it without re-running the test:

```
npm run load:report                       # latest run
LOAD_RUN_ID=20260730-171500 npm run load:report
```

6. Reading the results

- **Error rate** — first thing to check. High error rates invalidate response times.
- **p95 / p99** — the headline SLO numbers; averages hide the tail.
- **Throughput vs active VUs** — if throughput flattens while VUs keep rising, the system has saturated; the knee of that curve is the capacity limit.
- **Std dev** — large values indicate unstable response times (queueing, GC, locks).
- **Apdex** — single user-satisfaction score, `>= 0.94` is generally "good".
- **HTTP 5xx / failed requests** — server-side saturation signals.
- **SLO verdict** — `PASS`/`FAIL` computed from the `SLO_*` thresholds.

Why the error count is not inflated by one popup

An application error usually arrives as a blocking modal — for example `Oracle FLEXCUBE Message – FC_JSON_CLIENT_ERR_006`. The iteration that triggered it is aborted at the failing step, so the cleanup at the end of that flow (closing the confirmation and the task tab) never runs. Left alone, the modal stays on screen and every field underneath is unclickable, so each following iteration also fails — one real defect gets reported dozens of times.

Five mechanisms keep the numbers honest:

- **Cleanup inside the failed iteration** — as soon as a step throws, the harness dismisses every modal on screen, before the iteration is left. All stacked dialogs are cleared, not just the topmost one: FLEXCUBE often puts an error message on top of a session-timeout warning, and clearing only the top one leaves the screen just as blocked.
- **Pre-flight guard** — an iteration never starts on a screen that still has a modal on it. Anything left over is cleared first, so the first click of the new iteration is not measured against a stale overlay. If the popup cannot be cleared, the VU ends rather than logging failures caused by it.
- **Recovery hook** — after a failed iteration the flow's `recover` hook (`cbsResetToBaseline` for CBS) closes leftover task tabs, strips any leftover modal glass layers from the DOM (without reloading the page, which would kill the session), and then *proves* the screen is usable again by hit-testing the task search box (see below). If the session is genuinely lost, it signals `needsReLogin` and the harness re-runs login + FCR selection. Disable the whole mechanism with `RECOVER_ON_FAILURE=false`.
- **Circuit breaker** — if a VU still fails `MAX_CONSECUTIVE_FAILURES` (default `3`) iterations in a row, it is ended instead of continuing to log duplicates of the same unresolved problem. The report then shows a handful of real failures rather than hundreds of knock-on ones.
- **Cause attribution** — the modal text is captured before recovery and recorded as `APP DIALOG (blocking popup)`. This label is excluded from transaction statistics, so it never counts towards the error rate, but it appears in `errors.csv` and `error-details.txt` next to the failure it caused. That way the report names the actual FLEXCUBE error instead of only the Playwright timeout it produced.

The console makes the sequence explicit while the test runs:

```
[cashDeposit] VU 3 iteration 12 failed: transaction "02 Cash Deposit – Enter transaction" failed: locator.fill:
Timeout 10000ms exceeded
          app dialog: Oracle FLEXCUBE Message FC_JSON_CLIENT_ERR_006 : Couldn't find the Message for
[FC_JSON_CLIENT_ERR_006] OK
          screenshot: screenshots/cashDeposit-vu3-iter12.png
[cashDeposit] VU 3 closed 2 popup(s) left open by iteration 12
[cashDeposit] VU 3 recovered after iteration 12 (1 tab(s) closed)
```

If recovery itself cannot clear the screen, the VU is ended with the reason logged, which is the signal that the environment — not the script — needs attention.

Why "no dialog on screen" is not enough

Counting dialogs is not a reliable test of whether the screen works. When an Oracle JET dialog is torn down abnormally its modal glass layer (`div.oj-dialog-layer`) can be left behind: the dialog is gone, so every dialog-based check reports a clean screen, and Playwright still reports the task search box as visible and editable — but the transparent layer sits on top and swallows every click. The only visible symptom is `locator.click: Timeout 60000ms exceeded` on a control that looks perfectly fine.

Recovery therefore finishes by asking the browser what is *actually* at the centre of the task search box (`document.elementFromPoint`). If something else is on top, the leftover overlay elements (`.oj-dialog-layer`, `.oj-modal-glass`, etc.) are stripped from the DOM directly — **not** by reloading the page, which would invalidate the server-side session and force a full re-authentication.

If the session is genuinely lost (the popup was closed, or the server has timed out), recovery returns `needsReLogin: true` and the harness re-runs the full login + FCR selection flow, exactly as it does when `cbsLoggedOut` detects the login form. The log makes the sequence explicit:

```
[cashDeposit] VU 3 needs re-login after iteration 12: page closed
[cashDeposit] VU 3 session lost, re-authenticating (1)
```

Reproducing a stuck screen locally

Run one VU, headed, with the timeouts opened right up so the browser sits on the failing screen instead of tearing the run down while you inspect it. Turn recovery off so nothing is cleaned up before you can look at it:

```
LOAD_TEST=1 FLOWS=cashDeposit VUS=1 WORKERS=1 HEADLESS=false \
  ACTION_TIMEOUT_MS=600000 NAV_TIMEOUT_MS=600000 \
  FIELD_TIMEOUT_MS=300000 TEST_TIMEOUT_MS=7200000 \
  RECOVER_ON_FAILURE=false MAX_CONSECUTIVE_FAILURES=999 \
  DURATION_SEC=3600 RAMP_UP_SEC=0 \
  npx playwright test tests/cbs/cashDeposit.load.spec.js
```

Each VU prints its effective settings on startup, so the log states plainly which safety nets were armed:

```
[cashDeposit] VU 1 starting | recovery on | stop after 3 consecutive failures | action timeout 60000 ms | field wait
10000 ms
```

7. Report a run alongside server-side metrics

Client-side numbers alone are not a diagnosis. Capture CRM application server, DB and infrastructure metrics (CPU, memory, GC, thread pools, slow queries) over the same window and attach them to the run folder before sharing results.

8. Merging results from multiple VMs

When a single machine cannot generate enough VUs, distribute the load across multiple VMs. Each VM runs its own Playwright instance and produces a `load-results/<runId>/` directory with `raw/*.jsonl` sample files.

To merge all sources into one unified report:

```
# Copy each VM's run directory into load-results/ on the aggregation machine,
# then run:
npm run load:merge -- load-results/20260831-100000 load-results/20260831-100001

# Custom merged run ID:
npx cross-env LOAD_RUN_ID=merged-20260831 npm run load:merge -- \
  load-results/vm1-20260831-100000 load-results/vm2-20260831-100001
```

What the merge script does:

- Reads all `raw/*.jsonl` files from each source run directory.

- Offsets VU numbers per source VM (`vmIndex * 10000 + original VU`) so every VU is distinct in the aggregate — VU 1 from VM 1 and VU 1 from VM 2 do not collide.
- Copies and renames screenshots to match the offset VU numbers.
- Merges `run-metadata.json` from all sources (flows, VU counts, SLO thresholds).
- Runs the same aggregation pipeline as a single-VM report, producing `index.html`, `summary.json`, all CSVs, and `error-details.txt` in `load-results/<mergedRunId>/`.

The merged report is identical in format to a single-VM report — all percentiles, throughput, error breakdowns and time series are computed across the combined sample set.